



Dhaka International University

Lab Report details:

Lab Report Topic : **Write a code for Shortest Job First Algorithm using C/C++**

Lab Report No : 02

Course Title : Operating System

Course Code : 0613-302

Submitted By:	Submitted To:
S M Nabil Ausaf Roll: 01 Reg No : CS-D-91-23-124492	Md. Nazmus Sakib, Lecturer of CSE Department

Date of Submission:07.11.25

Lab No: 02

Lab Report Name

Implementation of Shortest Job First (SJF) CPU Scheduling Algorithm

Objectives

- ❖ To understand the concept of CPU scheduling and its importance in process management.
- ❖ To learn and implement the Shortest Job First (SJF) scheduling algorithm.
- ❖ To calculate **Average Waiting Time** and **Average Turnaround Time**.
- ❖ To compare the performance of SJF with other scheduling techniques (conceptually).

Background Study (with Example)

Definition:

Shortest Job First (SJF) is a **non-preemptive CPU scheduling algorithm** where the process with the **smallest burst time** is executed first. It minimizes the average waiting time for a set of processes.

Key Terms:

- **Burst Time (BT):** The time required by a process for execution.
- **Waiting Time (WT):** The total time a process spends waiting in the ready queue.
- **Turnaround Time (TAT):** The total time taken from process arrival to completion.

Formulas:

$$WT_i = TAT_i - BT_i$$

$$TAT_i = CompletionTime_i - ArrivalTime_i$$

Example:

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

Execution Order: P4 → P1 → P3 → P2

Average Waiting Time = 7.75 ms

Average Turnaround Time = 15.75 ms

Code

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      int tt = 0;
7      int min;
8      int d = 0;
9      int i, j;
10
11     float avg_tat = 0;
12     float avg_wt = 0;
13     float t_tat = 0;
14     float t_wt = 0;
15
16     cout << "Enter number of processes: ";
17     cin >> n;
18
19     int P[n], a[n], b[n], c[n], tat[n], wt[n];
20
21     cout << "Enter process sequence:\n";
22     for (i = 0; i < n; i++) {
23         cin >> P[i];
24     }
25
26     cout << "Enter arrival time:\n";
27     for (i = 0; i < n; i++) {
28         cin >> a[i];
29     }
30
31     cout << "Enter burst time:\n";
32     for (i = 0; i < n; i++) {
33         cin >> b[i];
34     }
```

```

35
36 // Sorting according to burst time (SJF)
37 for (i = 0; i < n; i++) {
38     for (j = i + 1; j < n; j++) {
39         if (b[i] > b[j]) {
40             swap(b[i], b[j]);
41             swap(a[i], a[j]);
42             swap(P[i], P[j]);
43         }
44     }
45 }
46
47 // Find process with minimum arrival time
48 min = a[0];
49 for (i = 0; i < n; i++) {
50     if (min > a[i]) {
51         min = a[i];
52         d = i;
53     }
54 }
55
56 tt = min;
57 c[d] = tt + b[d];
58 tt = c[d];
59
60 // Calculate completion times
61 for (i = 0; i < n; i++) {
62     if (i != d) {
63         if (a[i] > tt)
64             tt = a[i]; // CPU idle till process arrives
65         c[i] = tt + b[i];
66         tt = c[i];
67     }
68 }
69
70 // Display table
71 cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
72 for (i = 0; i < n; i++) {
73     tat[i] = c[i] - a[i];
74     wt[i] = tat[i] - b[i];
75     t_tat += tat[i];
76     t_wt += wt[i];
77
78     cout << "P[" << P[i] << "]\t" << a[i] << "\t" << b[i] << "\t"
79         << c[i] << "\t\t" << tat[i] << "\t\t" << wt[i] << endl;
80 }
81
82 avg_tat = t_tat / n;
83 avg_wt = t_wt / n;
84
85 cout << "\nAverage Turnaround Time: " << avg_tat << endl;
86 cout << "Average Waiting Time: " << avg_wt << endl;
87
88 return 0;
89 }
90

```

Input

```
Enter number of processes: 4
Enter process sequence:
1 2 3 4
Enter arrival time:
0 1 2 3
Enter burst time:
6 8 7 3
```

Output:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT-AT)	Waiting Time (TAT-BT)
P4	3	3	6	3	0
P1	0	6	12	12	6
P3	2	7	19	17	10
P2	1	8	27	26	18

Discussion

In this experiment, we implemented the **Shortest Job First (SJF)** scheduling algorithm. The results show that by executing the shortest process first, the **average waiting time** and **average turnaround time** are minimized compared to other algorithms like FCFS (First Come First Serve).

Advantages:

- Provides minimum average waiting time.
- Efficient for batch systems with known burst times.

Disadvantages:

- Not suitable for real-time systems.
- Requires prior knowledge of burst times.
- May cause **starvation** for longer processes.

Conclusion

The Shortest Job First (SJF) algorithm was successfully implemented and tested. It efficiently reduces average waiting and turnaround times by executing the shortest processes first. However, it requires knowing burst times in advance and may cause starvation for longer processes.